Practice #3: Heaps & Sorting حلول

خوارزميات وهياكل البيانات - تحليل كامل ومفصل

Q1: Trinary Heap Analysis (الشجرة الثلاثية)

الشجرة الثلاثية (Trinary Heap) هي شجرة كل عقدة غير ورقية فيها تدعم حتى 3 أبناء بدلاً من 2.

a) Array Representation (Indexing Mapping)

عند استخدام مصفوفة تبدأ من المؤشر 1 (1-based indexing):

- $\text{PARENT}(i) = \lfloor (i+1)/3 \rfloor$
- $\text{LEFT}(i) = 3i - 1$
- $\text{MIDDLE}(i) = 3i$
- $\text{RIGHT}(i) = 3i + 1$

b) Maximum number of nodes n with height h

المجموع الهندسي لأقصى عدد من العقد في شجرة كاملة بارتفاع h هو:

$$n = 1 + 3 + 3^2 + \ldots + 3^h = \sum_{k=0}^{h} 3^k = (3^{h+1} - 1)/(3 - 1) = (3^{h+1} - 1)/2$$

c) Height h of a trinary heap with n elements

من العلاقة أعلاه: $2n + 1 \leq 3^{h+1} \Rightarrow h + 1 = \lceil \log_3(2n+1) \rceil \Rightarrow h = \Theta(\log_3 n)$

d) Implementation of TRINARY-MAX-HEAPIFY & Running Time

```
TRINARY-MAX-HEAPIFY(A, i)
1  l = LEFT(i),   m = MIDDLE(i),   r = RIGHT(i)
2  largest = i
3  if l <= A.heap-size and A[l] > A[largest]
4      largest = l
5  if m <= A.heap-size and A[m] > A[largest]
6      largest = m
7  if r <= A.heap-size and A[r] > A[largest]
8      largest = r
9  if largest != i
10     exchange A[i] with A[largest]
11     TRINARY-MAX-HEAPIFY(A, largest)
```

التحليل الزمني: المقارنة بين الأبناء الثلاثة تستغرق وقتاً ثابت $O(1)$. في أسوأ حالة ننتقل إلى أحد الأبناء، مما ينقص الارتفاع بمقدار 1. العلاقة التكرارية: $T(n) \leq T(n/3) + O(1) \Rightarrow T(n) = O(\log_3 n)$.

e) Implementation of TRINARY-HEAP-EXTRACT-MAX

```
TRINARY-HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1 error "heap underflow"
2  max = A[1]
3  exchange A[1] with A[A.heap-size]
4  A.heap-size = A.heap-size - 1
5  TRINARY-MAX-HEAPIFY(A, 1)
6  return max
```

التحليل الزمني: عمليات التبديل وتحديث الحجم تستغرق $O(1)$. إعادة بناء الهيب تستدعي TRINARY-MAX-HEAPIFY التي تستغرق $O(\log_3 n)$. إذن التعقيد الكلي هو $O(\log_3 n)$.

f) Implementation of TRINARY-HEAP-INCREASE-KEY

```
TRINARY-HEAP-INCREASE-KEY(A, i, key)
1  if key < A[i] error "new key is smaller than current key"
2  A[i] = key
3  while i > 1 and A[PARENT(i)] < A[i]
4       exchange A[i] with A[PARENT(i)]
5       i = PARENT(i)
```

التحليل الزمني: في أسوأ حالة، يصعد العنصر من المستوى الأخير إلى الجذر على طول ارتفاع الشجرة. بالتالي الوقت الكلي هو $O(h) = O(\log_3 n)$.

g) Implementation of TRINARY-MAX-HEAP-INSERT

```
TRINARY-MAX-HEAP-INSERT(A, key)
1  A.heap-size = A.heap-size + 1
2  A[A.heap-size] = -∞
3  TRINARY-HEAP-INCREASE-KEY(A, A.heap-size, key)
```

التحليل الزمني: زيادة الحجم يستغرق $O(1)$، واستدعاء زيادة المفتاح يستغرق $O(\log_3 n)$. الوقت الكلي هو $O(\log_3 n)$.

Q2: Algorithms & Complexity Applications

a) Merge k sorted lists into one sorted list in $O(n \log k)$ time

```
MERGE-K-SORTED-LISTS(L_1...L_k, k, n)
1  FinalList = array of size n
2  A = empty min-heap array of size k
3  for i = 1 to k
4       insert first element of L_i into A with tag storing list index i
5  BUILD-MIN-HEAP(A)
6  for j = 1 to n
7       min_elem = EXTRACT-MIN(A)
8       FinalList[j] = min_elem.value
9       let m = min_elem.list_index
10      if list L_m has a next element
11          INSERT-MIN-HEAP(A, next element from L_m)
12 return FinalList
```

التحليل الزمني: بناء Min-Heap بحجم k يستغرق $O(k)$. تتكرر الحلقة n مرة، وتنفذ في كل دورة عملية استخراج وإدراج على هيب حجمه k وتكلفتهما $O(\log k)$. التعقيد الكلي: $O(k) + O(n \log k) = O(n \log k)$.

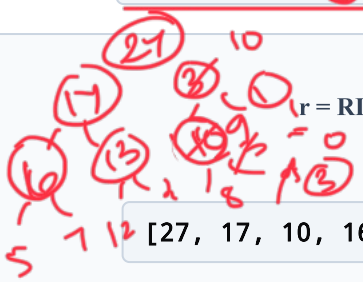
b) Sort unsorted array A by splitting into k sorted lists first

```
SORT-BY-SPLITTING(A, n, k)
1  Split A into k sub-arrays, each of size approx (n/k)
2  For each sub-array i from 1 to k:
3       Sort sub-array i using MergeSort or HeapSort
4  Result = MERGE-K-SORTED-LISTS(sub-arrays, k, n)
5  return Result
```

التحليل الزمني: ترتيب k مصفوفات فرعية كل منها بحجم (n/k) يحتاج وقت $k \times O((n/k)\log(n/k)) = O(n \log(n/k))$. دمج هذه المصفوفات يلزمه $O(n \log k)$ من الفقرة (a). إجمالي الوقت: $O(n \log(n/k)) + O(n \log k) = O(n (\log n - \log k + \log k)) = O(n \log n)$.

Q3: Operation of MAX-HEAPIFY(A, 3)

المصفوفة المعطاة: `A = [27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0]`



الخطوة 1: عند المؤشر i = 3 (القيمة A[3] = 3)

- الأبناء هم: l = LEFT(3) = 6 ⇒ A[6] = 10 و r = RIGHT(3) = 7 ⇒ A[7] = 1
- العنصر الأكبر هو A[6] = 10 (لأن 10 > 3).
- يتم التبديل بين A[3] و A[6].
- تصبح المصفوفة: `[27, 17, 10, 16, 13, 3, 1, 5, 7, 12, 4, 8, 9, 0]`
- استدعاء تكراري: MAX-HEAPIFY(A, 6).

الخطوة 2: عند المؤشر i = 6 (القيمة A[6] = 3)

- الأبناء هم: l = LEFT(6) = 12 ⇒ A[12] = 8 و r = RIGHT(6) = 13 ⇒ A[13] = 9
- العنصر الأكبر هو A[13] = 9 (لأن 9 > 8 > 3).
- يتم التبديل بين A[6] و A[13].
- تصبح المصفوفة: `[27, 17, 10, 16, 13, 9, 1, 5, 7, 12, 4, 8, 3, 0]`
- استدعاء تكراري: MAX-HEAPIFY(A, 13).

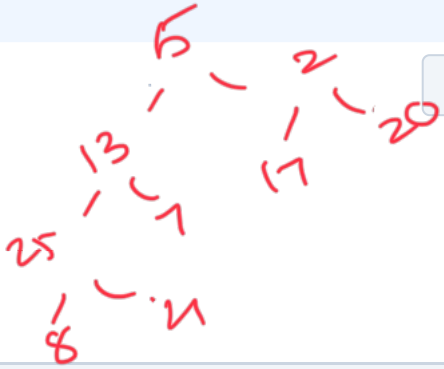
الخطوة 3: عند المؤشر i = 13

ليس لديه أبناء داخل الهيب (i = 26 > heap-size). تتوقف الخوارزمية.

النتيجة النهائية للمصفوفة:

`A = [27, 17, 10, 16, 13, 9, 1, 5, 7, 12, 4, 8, 3, 0]`

Q4: Full Execution of HEAPSORT(A)



المصفوفة الأولية (n = 9): `A = [5, 13, 2, 25, 7, 17, 20, 8, 4]`

1) بعد إجراء BUILD-MAX-HEAP(A):

`A = [25, 13, 20, 8, 7, 17, 2, 5, 4]`

2) تتبع دورات الاستخراج وإعادة التعديل (Loop Iterations):

الدورة (i)	عملية التبديل مع الجذر	حالة المصفوفة بعد MAX-HEAPIFY
i = 9	تبديل A[1] (25) مع A[9] (4)	[20, 13, 17, 8, 7, 4, 2, 5, 25]
i = 8	تبديل A[1] (20) مع A[8] (5)	[17, 13, 5, 8, 7, 4, 2, 20, 25]
i = 7	تبديل A[1] (17) مع A[7] (2)	[13, 8, 5, 2, 7, 4, 17, 20, 25]
i = 6	تبديل A[1] (13) مع A[6] (4)	[8, 7, 5, 2, 4, 13, 17, 20, 25]
i = 5	تبديل A[1] (8) مع A[5] (4)	[7, 4, 5, 2, 8, 13, 17, 20, 25]
i = 4	تبديل A[1] (7) مع A[4] (2)	[5, 4, 2, 7, 8, 13, 17, 20, 25]
i = 3	تبديل A[1] (5) مع A[3] (2)	[4, 2, 5, 7, 8, 13, 17, 20, 25]
i = 2	تبديل A[1] (4) مع A[2] (2)	[2, 4, 5, 7, 8, 13, 17, 20, 25]

Q5: Operation of MAX-HEAP-INSERT(A, 10)

المصفوفة الأصلية (heap-size = 12): `A = [15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1]`

1. زيادة الحجم: heap-size = 13 وإسناد القيمة المبدئية $-\infty$ = A[13].
2. استدعاء HEAP-INCREASE-KEY(A, 13, 10) وضع القيمة 10 في الخانة 13.
3. مقارنة مع الأب A[6] = 8 ⇒ PARENT(13) = 6. بما أن 10 > 8 يتم التبديل.

   المصفوفة: `[15, 13, 9, 5, 12, 10, 7, 4, 0, 6, 2, 1, 8]`
4. مقارنة مع الأب A[3] = 9 ⇒ PARENT(6) = 3. بما أن 10 > 9 يتم التبديل.

   المصفوفة: `[15, 13, 10, 5, 12, 9, 7, 4, 0, 6, 2, 1, 8]`
5. مقارنة مع الأب A[1] = 15 ⇒ PARENT(3) = 1. بما أن 10 < 15 تتوقف الخوارزمية.

النتيجة النهائية: `A = [15, 13, 10, 5, 12, 9, 7, 4, 0, 6, 2, 1, 8]`

Q6: Operation of HEAP-EXTRACT-MAX(A)

المصفوفة الأصلية (heap-size = 12): `A = [15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1]`

1. حفظ القيمة العظمى max = A[1] = 15.
2. تبديل العنصر الأول A[1] مع العنصر الأخير A[12] = 1، ثم تقليل الحجم إلى 11.
3. إجراء MAX-HEAPIFY(A, 1) لإصلاح الهيب:
   - عند i = 1: الأبناء A[2]=13, A[3]=9. الأكبر هو 13. نتبادل مع A[2].

     المصفوفة: `[13, 1, 9, 5, 12, 8, 7, 4, 0, 6, 2]`
   - عند i = 2: الأبناء A[4]=5, A[5]=12. الأكبر هو 12. نتبادل مع A[5].

     المصفوفة: `[13, 12, 9, 5, 1, 8, 7, 4, 0, 6, 2]`
   - عند i = 5: الأبناء A[10]=6, A[11]=2. الأكبر هو 6. نتبادل مع A[10].

     المصفوفة: `[13, 12, 9, 5, 6, 8, 7, 4, 0, 1, 2]`

العنصر المستخرج: 15 | **المصفوفة النهائية:** `A = [13, 12, 9, 5, 6, 8, 7, 4, 0, 1, 2]`

Q7: Analysis of BUILD-MAX-HEAP' Variation

?a) Do BUILD-MAX-HEAP and BUILD-MAX-HEAP' always create the same heap

الإجابة: NO (لا يُنتجان نفس الهيب دائماً).

مثال مضاد (Counter-Example): نأخذ المصفوفة المدخلة: `A = [1, 2, 3]`

- باستخدام الخوارزمية الأصلية BUILD-MAX-HEAP(A) نحصل على: `[3, 2, 1]`
- باستخدام البديلة BUILD-MAX-HEAP'(A) نقوم بإدراج العناصر متتالياً:
  1. إدراج 1: [1]
  2. إدراج 2: [2, 1]
  3. إدراج 3: [3, 1, 2]

بما أن [3, 1, 2] ≠ [3, 2, 1]، الخوارزميتين تعطيان ناتجين مختلفين.

b) Show that in worst case BUILD-MAX-HEAP' requires Θ(n log n) time